

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a
Time	$O(b^d)$	$O(b^{1+1/C^*/\epsilon})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+1/C^*/\epsilon})$	$O(bm)$	$O(b\epsilon)$	$O(bd)$
Optimal?	Yes ^c	Yes	No	No	Yes ^c

Uniform Search Methods 不知情搜索方法

1.广度优先搜索 Breadth-First Search

Breadth First Search

- Strategy:
 - Expand a shallowest node first

- Implementation:
 - Frontier is a FIFO queue



2.深度优先搜索 Depth-First Search

Depth First Search

- Strategy:
 - Expand a deepest node first

- Implementation:
 - Frontier is a LIFO stack



3.Iterative Deepening IDDFS 迭代加深搜索

想法:利用 BFS 的时间/浅解优势和 DFS 的空间优势=>运行**深度限制**为 1,2,3, ..., 的 DFS 直到找到解决办法(在有深度限制情况下进行深度优先检测)

这不浪费?大多数工作发生在最低级别搜索中=>还不错!

时间/空间复杂度与 DFS 相同, 但具有完备性和最优性。

4.Uniform-Cost Search UCS 一致代价搜索

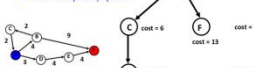
为一种 Cost-Sensitive Search 考虑代价的搜索: 可以找到最小代价路径(以 BFS 为基础加上搜索代价, 未进行预测)。

Uniform-cost Search

- Cost-Sensitive Search

- Strategy:
 - Expand a cheapest node first

- Implementation:
 - Frontier is a priority queue



耗时 $O(b^{1+C^*/\epsilon})$ (有效深度指数)/空间 $O(b^{1+C^*/\epsilon})$ /假设最佳解决方案的成本是有限的, 最小弧线成本是正的, 则具有完备性! /具有最优性!

Informed Search Methods 启发式搜索/信息搜索

UCS 要探索所有节点=>bad, 浪费很多时间和空间

-Heuristics 启发式算法

A heuristic is: a function that estimates how close a state is to a goal / Designed for a **particular** search problem (在实际计算代价之前, 先行进行预测, 以此不必检测所有 nodes, 节约时间和空间. 那么需要设计一个用于预测的启发式函数) / Examples: Manhattan distance 曼哈顿距离, 即上下左右四个方向情况下的起点与终点横竖距离绝对值的相加) Euclidean distance 欧几里得距离(即直线距离) for pathing

-Greedy Best-First Search 贪食最佳优先搜索

通过设计 a node that **you think is closest** to a goal state (并不是实际上最近的, 而是通过你的启发式算法 Heuristics 算出来的你认为最近的) (每一步选择时并不会考虑这一步之前的代价, 只会考虑在 DFS 的 fringe 即边缘里面选择代价最小的一个 h(n))。

贪心搜索可以快速找到最优解, 但可能直接得到解/错解。

-不具备完备性和最优性

最坏情况下的时间和空间复杂度为 O (bⁿm)

A* Search

Combining UCS and Greedy <= **Uniform-cost** orders by path cost, or backward cost **g(n)** 已经消耗的代价(真实的代价) / **Greedy** orders by goal proximity, or forward cost **h(n)** 预测的从当前位置到目标所需的代价(非真实代价)

=>A* Search orders by the sum: **f(n) = g(n) + h(n)**

不是遇到 goal node 之后就停下, 而是找到最小代价的到达 goal node 的路径后才停下。

最优性: 如果 the heuristic is **admissible**, the **tree-search** version of A* is **optimal**

A heuristic h is admissible (optimistic) iif: **0 ≤ h(n) ≤ h*(n)**

启发式函数计算的代价要能保证不超过真实代价

如果 the heuristic is **consistent**, the **graph-search** version A* is **optimal** (比 admissible 要求更高, consistent=>admissible. 树搜索可以遍历每条路线, 但是若有两条路线 sacg 和 sbcg, 图搜索在选择遍历 sacg 路径后已经遍历过 c 结点, 因此遍历 sbc 想扩展 c 结点时无法再次在此处扩展遍历/c 会被 skip), 这会导致找到的路线必定是搜索的第一条)

Consistency: heuristic “arc” cost ≤ actual cost for each arc **h(A) - h(C) ≤ cost(A to C)** 不仅需要到终点的启发式函数预估代价小于实际代价, 还需要每一步预估代价都小于实际代价。

A* 具有完备性! A* 对于任何一致性启发式算法都最优, 无其他最优算法能保证扩展更少节点。

不像 Greedy 每次都只向单次最优的方向探索, A* 也会探索别的方向=>为了保证最终结果的最优性, 但不会像 UCS 一样完全平等地探索每一个方向

Adversarial Search 敌对搜索

Minimax Search 最大最小值搜索

一种 game tree/玩家交替出招/minimax 基于一种假设, 即对手也会进行最优操作并总是会 Proof:

- Suppose a suboptimal solution node n with solution value $C' > C^*$ is about to be expanded (where C^* is optimal);

- Let n^* be an optimal solution node (not yet discovered);

There must be some node n' that is currently in the frontier and on the path to n^* ;

- We have $g(n') = C' > C^* = g(n^*) \geq g(n') + h(n')$;

- But then, n' should be expanded first (contradiction).

$h(n) \leq c(n, a, n') + h(n')$

Strategy: Expand a **shallowest** node first

Implementation: Frontier is a **FIFO queue** 先进先出堆栈

Properties: 处理最浅解以上的所有节点/令最浅解深度为 s, 搜索时间空间均为 O(b^sm)/具有完备性, 因为解一定存在/当且仅当 costs 全为 1 时具有最优性。

Strategy: expand a **deepest** node first

Implementation: Frontier is a **LIFO stack** 后进先出堆栈

Properties 属性: 先拓展左边的树节点/可以处理整棵树/如果深度 m 有限, 搜索时间 O(b^mm)/空间 O(bm)/完备性:m 可以是无限的, 所以仅当我们避免循环时完备/不是 optimal 最优的, 始终找到 leftmost

方案, 不考虑深度或成本

3.Iterative Deepening IDDFS 迭代加深搜索

想法:利用 BFS 的时间/浅解优势和 DFS 的空间优势=>运行**深度限制**为 1,2,3, ..., 的 DFS 直到找到解决办法(在有深度限制情况下进行深度优先检测)

这不浪费?大多数工作发生在最低级别搜索中=>还不错!

时间/空间复杂度与 DFS 相同, 但具有完备性和最优性。

4.Uniform-Cost Search UCS 一致代价搜索

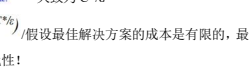
为一种 Cost-Sensitive Search 考虑代价的搜索: 可以找到最小代价路径(以 BFS 为基础加上搜索代价, 未进行预测)。

Uniform-cost Search

- Cost-Sensitive Search

- Strategy:
 - Expand a cheapest node first

- Implementation:
 - Frontier is a priority queue



耗时 $O(b^{1+C^*/\epsilon})$ (有效深度指数)/空间 $O(b^{1+C^*/\epsilon})$ /假设最佳解决方案的成本是有限的, 最小弧线成本是正的, 则具有完备性! /具有最优性!

Informed Search Methods 启发式搜索/信息搜索

UCS 要探索所有节点=>bad, 浪费很多时间和空间

-Heuristics 启发式算法

A heuristic is: a function that estimates how close a state is to a goal / Designed for a **particular** search problem (在实际计算代价之前, 先行进行预测, 以此不必检测所有 nodes, 节约时间和空间. 那么需要设计一个用于预测的启发式函数) / Examples: Manhattan distance 曼哈顿距离, 即上下左右四个方向情况下的起点与终点横竖距离绝对值的相加) Euclidean distance 欧几里得距离(即直线距离) for pathing

-Greedy Best-First Search 贪食最佳优先搜索

通过设计 a node that **you think is closest** to a goal state (并不是实际上最近的, 而是通过你的启发式算法 Heuristics 算出来的你认为最近的) (每一步选择时并不会考虑这一步之前的代价, 只会考虑在 DFS 的 fringe 即边缘里面选择代价最小的一个 h(n))。

贪心搜索可以快速找到最优解, 但可能直接得到解/错解。

-不具备完备性和最优性

最坏情况下的时间和空间复杂度为 O (bⁿm)

A* Search

Combining UCS and Greedy <= **Uniform-cost** orders by path cost, or backward cost **g(n)** 已经消耗的代价(真实的代价) / **Greedy** orders by goal proximity, or forward cost **h(n)** 预测的从当前位置到目标所需的代价(非真实代价)

=>A* Search orders by the sum: **f(n) = g(n) + h(n)**

不是遇到 goal node 之后就停下, 而是找到最小代价的到达 goal node 的路径后才停下。

最优性: 如果 the heuristic is **admissible**, the **tree-search** version of A* is **optimal**

A heuristic h is admissible (optimistic) iif: **0 ≤ h(n) ≤ h*(n)**

启发式函数计算的代价要能保证不超过真实代价

如果 the heuristic is **consistent**, the **graph-search** version A* is **optimal** (比 admissible 要求更高, consistent=>admissible. 树搜索可以遍历每条路线, 但是若有两条路线 sacg 和 sbcg, 图搜索在选择遍历 sacg 路径后已经遍历过 c 结点, 因此遍历 sbc 想扩展 c 结点时无法再次在此处扩展遍历/c 会被 skip), 这会导致找到的路线必定是搜索的第一条)

Consistency: heuristic “arc” cost ≤ actual cost for each arc **h(A) - h(C) ≤ cost(A to C)** 不仅需要到终点的启发式函数预估代价小于实际代价, 还需要每一步预估代价都小于实际代价。

A* 具有完备性! A* 对于任何一致性启发式算法都最优, 无其他最优算法能保证扩展更少节点。

不像 Greedy 每次都只向单次最优的方向探索, A* 也会探索别的方向=>为了保证最终结果的最优性, 但不会像 UCS 一样完全平等地探索每一个方向

Adversarial Search 敌对搜索

Minimax Search 最大最小值搜索

一种 game tree/玩家交替出招/minimax 基于一种假设, 即对手也会进行最优操作并总是会 Proof:

- Suppose a suboptimal solution node n with solution value $C' > C^*$ is about to be expanded (where C^* is optimal);

- Let n^* be an optimal solution node (not yet discovered);

There must be some node n' that is currently in the frontier and on the path to n^* ;

- We have $g(n') = C' > C^* = g(n^*) \geq g(n') + h(n')$;

- But then, n' should be expanded first (contradiction).

$h(n) \leq c(n, a, n') + h(n')$

采取最小利己的策略。计算每个节点的 minimax 值。



在对手控制下的操作 我们取得最小 min; 而在我们最小控制下的我们取最大 max。

Minimax 最终得到结果是对抗理性(最优)对手的最佳可实现效用。

在实现时, minimax 与 DFS 有些类似, 与 DFS 采取相同的顺序来计算节点的值, 从左至叶子节点开始并向右迭代。

更准确的说, 它在游戏树中使用了后序遍历 (postorder traversal)。

Properties: 当面对最优对手时才有最优性, 其他时候未必。/搜索时间 O(b^mm)/空间 O(bm), 与 DFS 相同/m 有限时有完备性。

Alpha-Beta Pruning 剪枝

Minimax 虽然简单直观最优, 但时间效率差, 为改善这点, 进行 alpha-beta 剪枝。

Worked example



在这个例子中, 当访问到第二个 MAX 第一个子节点值为 9 时, 无论后面的子节点如何取值, 这个最大值节点一定大于等于 9, 而第一个节点 MAX=8, 所以第一个 MIN 节点候选值为 8 和大于等于 9, 只能取 8, 因此我们不再需要查看第二个节点的第二个子节点了。同理, 第三个 MAX=2, 因此第二个 MIN 节点一定小于等于 2, 而 MAX 根节点的候选值为 8 和小于等于 2, 则无论最右 MAX 取多少, 根节点都一定取 8。这正是为什么我们可以直接抛弃子节点, 从而对这棵搜索树进行剪枝。这可能致中间值错误, 但结果始终正确。

剪枝的**有效性** efficiency 高度依赖于检查状态的顺序: 在完美的情况下, alpha-beta 只需要检查 O(b^m(m/2))个节点(时间复杂度); 随机情况下, 也只有 O(b^m(3m/4))的时间复杂度

例图中, 倒三角是 MIN, 正三角是 MAX。

Constraint Satisfaction Problems 约束满足问题

约束满足问题指的是针对给定的一组变量及其需要满足的一些限制或一些约束条件, 找出符合这些满足这些约束关系的一个解、全部解或是最优解。在平内容!

A CSP consists of three components:

A set of **variables** X={X1,...,Xn} 变量

A set of **domains** D={D1,...,Dn} 域

A set of **constraints** C 约束条件

状态是通过将域的值赋值给部分或全部变量来定义的。

Backtracking Search 回溯搜索

回溯搜索是求解 CSP 的基本 uninformed 算法(无信息)。

回溯= DFS + 变量排序+逆例失败 Backtracking search is the **depth-first search** with two

improvements:1.One variable at a time 一次一个变量 2.Backtrack when a variable has no legal values 当变量没有合法值时回溯。左就是一个例子。每次针对一个变量, 无法继续拓展且未找到解就回溯到选择的地方。

Inference in CSPs:

Inference 推理:约束传播, 减少一个变量的合法值的数量。约束传播可以与搜索交织在一起, 或者作为一个预处理步骤。关键思想是局部一致性 local consistency。

1.Forward checking 前向检查

前向检查:划掉添加到现有赋值时违反约束的值; 将信息从已赋值变量传播到未赋值变量; 跟踪未分配变量的域, 当任何变量没有合法值时终止搜索。如右上, 当 Q1 取在 (1,1) 时, 就知道标蓝色的地方之后都不能取 (Qn 表示第 n 行取值)。其他仍按照回溯搜索的规则, 最终得到这个, 这就是使用前向检查时得到结果。

2.Arc Consistency 弧相容

An arc X->Y is **consistent** if for **every** x there is **some** y which could be assigned without violating a constraint 对于每个 X 都有一个 Y 可以在不违反约束的情况下被赋值。

前向检查:强制指向每个新赋值的弧的一致性。

在确定 Arc Consistency 的时候只在乎这条 arc, 并不在乎整图是否能成立。牢记对 every x in the tail,都有 some y in the head! 注意在删去操作之后, 需要重新对之前做过的涉及该删去点的 arc 进行会检查

重新涉及因为之前的 arc 可能会因删去而变成(in)consistent。如果在一次删去操作后, 有一个地点没有可赋值了, 那么终止。弧相容比前向检查更早检测到故障。可作为预处理器也可在每次赋值后运行。

左图是弧相容的 n 皇后结果。其第一步和前向检查第一步一样, 但第二步是在第一步基础上继续检查, 直到发现故障/结果。

Arc-Consistency Algorithm (AC-3): 应用 AC-3 后: 要么每个弧都是弧一致的, 要么某个变量有一个空域, 说明 CSP 无法求解。

假设一个 CSP 有 n 个变量, d 个域大小和 c 个二进制约束; 每个弧(Xk,Xi)可插入 d 次; 检查弧的一致性需要 O(d²)时间,所以总的最坏情况时间是 O(cd⁴)3。

局限性: 运用弧一致后, 可能有多于一个/没有解决方案。可以帮助更快接近答案/排除错误, 但仍要和回溯搜索一起用, 无法仅凭弧一致解决 CSP 问题。

K-consistency K 一致性

1-Consistency (Node Consistency): 每个单个节点的域有一个满足该节点一元约束的值

2-Consistency (弧一致性):对于每一对节点, 任何对一个节点的一致分配都可以扩展到另一个节点/ k-consistency:对于每 k 个节点, 任何对 k-1 的一致分配都可以扩展到 kth 节点。k 值越大, 计算代价越大。

Strong n-consistency means we can solve **without backtracking** 但工作量非常大, 不如用前向检查等来进行一些 tradeoffs

3.The Structure of Problems-Tree-Structured CSPs

也可以通过改变问题结构来简化问题, 把图结构改成树结构

首先, 在 CSP 的约束图中任选一个节点来作为树的根节点 将树中的所有无向边转换为指向根节点反方向的有向边。然后将得到的有向无图线性化 (linearize) (或拓扑排序

在对手控制下的操作 我们取得最小 min; 而在我们最小控制下的我们取最大 max。

Minimax 最终得到结果是对抗理性(最优)对手的最佳可实现效用。

在实现时, minimax 与 DFS 有些类似, 与 DFS 采取相同的顺序来计算节点的值, 从左至叶子节点开始并向右迭代。

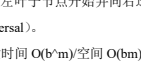
更准确的说, 它在游戏树中使用了后序遍历 (postorder traversal)。

Properties: 当面对最优对手时才有最优性, 其他时候未必。/搜索时间 O(b^mm)/空间 O(bm), 与 DFS 相同/m 有限时有完备性。

Alpha-Beta Pruning 剪枝

Minimax 虽然简单直观最优, 但时间效率差, 为改善这点, 进行 alpha-beta 剪枝。

Worked example



在这个例子中, 当访问到第二个 MAX 第一个子节点值为 9 时, 无论后面的子节点如何取值, 这个最大值节点一定大于等于 9, 而第一个节点 MAX=8, 所以第一个 MIN 节点候选值为 8 和大于等于 9, 只能取 8, 因此我们不再需要查看第二个节点的第二个子节点了。同理, 第三个 MAX=2, 因此第二个 MIN 节点一定小于等于 2, 而 MAX 根节点的候选值为 8 和小于等于 2, 则无论最右 MAX 取多少, 根节点都一定取 8。这正是为什么我们可以直接抛弃子节点, 从而对这棵搜索树进行剪枝。这可能致中间值错误, 但结果始终正确。

剪枝的**有效性** efficiency 高度依赖于检查状态的顺序: 在完美的情况下, alpha-beta 只需要检查 O(b^m(m/2))个节点(时间复杂度); 随机情况下, 也只有 O(b^m(3m/4))的时间复杂度

例图中, 倒三角是 MIN, 正三角是 MAX。

Constraint Satisfaction Problems 约束满足问题

约束满足问题指的是针对给定的一组变量及其需要满足的一些限制或一些约束条件, 找出符合这些满足这些约束关系的一个解、全部解或是最优解。在平内容!

A CSP consists of three components:

A set of **variables** X={X1,...,Xn} 变量

A set of **domains** D={D1,...,Dn} 域

A set of **constraints** C 约束条件

状态是通过将域的值赋值给部分或全部变量来定义的。

Backtracking Search 回溯搜索

回溯搜索是求解 CSP 的基本 uninformed 算法(无信息)。

回溯= DFS + 变量排序+逆例失败 Backtracking search is the **depth-first search** with two

improvements:1.One variable at a time 一次一个变量 2.Backtrack when a variable has no legal values 当变量没有合法值时回溯。左就是一个例子。每次针对一个变量, 无法继续拓展且未找到解就回溯到选择的地方。

Inference in CSPs:

Inference 推理:约束传播, 减少一个变量的合法值的数量。约束传播可以与搜索交织在一起, 或者作为一个预处理步骤。关键思想是局部一致性 local consistency。

1.Forward checking 前向检查

前向检查:划掉添加到现有赋值时违反约束的值; 将信息从已赋值变量传播到未赋值变量; 跟踪未分配变量的域, 当任何变量没有合法值时终止搜索。如右上, 当 Q1 取在 (1,1) 时, 就知道标蓝色的地方之后都不能取 (Qn 表示第 n 行取值)。其他仍按照回溯搜索的规则, 最终得到这个, 这就是使用前向检查时得到结果。

2.Arc Consistency 弧相容

An arc X->Y is **consistent** if for **every** x there is **some** y which could be assigned without violating a constraint 对于每个 X 都有一个 Y 可以在不违反约束的情况下被赋值。

前向检查:强制指向每个新赋值的弧的一致性。

在确定 Arc Consistency 的时候只在乎这条 arc, 并不在乎整图是否能成立。牢记对 every x in the tail,都有 some y in the head! 注意在删去操作之后, 需要重新对之前做过的涉及该删去点的 arc 进行会检查

重新涉及因为之前的 arc 可能会因删去而变成(in)consistent。如果在一次删去操作后, 有一个地点没有可赋值了, 那么终止。弧相容比前向检查更早检测到故障。可作为预处理器也可在每次赋值后运行。

